# Lifeverse — DevOps / MLOps Engineer Technical Assessment

## 1. Assessment overview

You will build, package and operate a small ML-backed web application, then diagnose a short production incident. The focus is DevOps: Docker, CI/CD, configuration, observability, security and operational reliability. You will also show working MLOps fundamentals (model packaging, inference behaviour and model versioning) and connect a minimal frontend to a backend API.

| ITEM | DETAIL |
|---|---|
| Role | DevOps / MLOps Engineer |
| Level | Intermediate (semi-senior) |
| Suggested effort | 5–7 hours, maximum one working day |
| Troubleshooting exercise | Approximately 30–45 minutes (included in the total) |
| Cost and access | No paid cloud account or Lifeverse internal access is required |

GROUND RULES

- You may use official documentation and AI coding assistants.
- You must be able to explain every part of your implementation. After submission, you will walk us through your solution in a 15–20 minute follow-up interview.
- Prioritise a reliable, understandable baseline. Optional extensions (section 5) do not compensate for missing mandatory requirements.

### What you will build

A user submits an input through a simple frontend, the backend API runs it through a packaged model, and the user sees an understandable prediction. You then package the service in Docker, automate checks with CI, make configuration explicit and add basic operational signals.

EXPECTED FLOW:

User → frontend → backend API → packaged model → prediction response

### Model and data

- **Default:** use a small pre-trained model, such as a lightweight image-classification model. You can browse and load available models from the PyTorch Vision pre-trained models catalogue.
- **Training is not required.** You do not need to train a model from scratch.
- **Alternative tasks:** an equivalent lightweight ML task is acceptable only if you agree it with us before you start. If you choose an audio-based task, the Speech Activity Detection datasets on Kaggle are a suitable source of sample inputs. Use the dataset only as far as your chosen task needs it.

## 2. Required implementation

All items in this section are mandatory unless marked *where feasible* or *optional*. Areas B, C and D are the priority of this assessment.

### A. Frontend and backend application

1. **Frontend.** Build a small frontend with a tool you know, such as Streamlit or a lightweight web framework. A user must be able to provide an input, submit it and view the prediction, plus any useful confidence or label information.
2. **Backend API.** Implement the API with FastAPI or Flask. Keep responsibilities separated: the frontend must call the API, not load the model directly.
3. **Endpoints.** Provide `GET /health` and `POST /predict`, or equivalent endpoints that you document.
4. **Error handling.** Validate inputs. Show clear messages for invalid input, an unavailable API and inference errors.
5. **Example and limitations.** Include at least one example of a successful prediction. Explain the model's expected inputs and its limitations.

### B. Docker and runtime configuration (PRIORITY)

1. Provide a `Dockerfile` and dependency definitions that produce a repeatable build.
2. Document how to build and run the application locally.
3. Use environment variables or a configuration file for runtime settings such as API URL, port and model version. Do not hard-code environment-specific values throughout the code.
4. Keep secrets out of source control and out of container images. Include a suitable `.dockerignore`.
5. *Where feasible:* run the service as a non-root user and define a sensible container health check.

### C. CI/CD pipeline (PRIORITY)

1. Add a GitHub Actions workflow, or an equivalent, that runs on every push and pull request.

2. The pipeline must install dependencies, run automated tests and run a basic code-quality check.
3. The pipeline must build the Docker image.
4. Explain in writing:
   - what would trigger a deployment;
   - how configuration differs between environments;
   - how you would roll back a faulty release.

*Optional but encouraged:* publish the image or deploy it to a cloud environment.

### D. Observability and operational reliability (PRIORITY)

1. **Logs.** Log startup, request failures and inference errors. Do not log secrets or unnecessary user input.
2. **Health.** Expose a health endpoint. If relevant, distinguish service health from model readiness.
3. **Timing.** Record or return basic request timing. Explain how you would investigate elevated latency or repeated failures.
4. **Risks.** Document at least three operational risks and how you would detect or mitigate each one.

### E. MLOps fundamentals

1. **Packaging and loading.** Package and load the model reproducibly. Do not download or reload the model on every request, unless you explain why you chose to.
2. **Identification.** State the model name and version in your documentation and, where appropriate, in the API response or logs.
3. **Lifecycle.** Explain how you would update the model, validate the new version, deploy it safely and roll back if prediction quality or latency regresses.
4. **Inference testing.** Describe at least one test of inference behaviour that goes beyond checking that the endpoint returns HTTP 200.

## 3. Troubleshooting exercise (30–45 minutes)

Review the incident below and write a diagnosis and a prioritised recovery plan. Put your answer in a short section of your README or in a separate `INCIDENT_NOTES.md`. You do not need access to a real cloud account.

> **Incident.** A new release was deployed. The frontend loads, but predictions now fail intermittently. The CI workflow passed. Container logs show occasional connection errors from the frontend to the API, some requests return HTTP 500, and the API container restarts after deployment. The service worked in the previous release.

YOUR TASKS

1. List the first checks you would perform and the evidence you would collect.
2. Identify plausible causes across configuration, networking, container health and resources, application logs and model loading. Do not settle on a single cause without evidence.
3. Describe how you would restore service safely, including when you would roll back.
4. Propose one or two changes to CI/CD or observability that would reduce the likelihood or impact of a recurrence.

There is no hidden root cause to guess. We are looking for structured debugging, clear prioritisation, safe recovery and reasoning based on evidence.

## 4. Deliverables

Submit all of the following. Use this list as your final checklist.

- [ ] **Source repository** containing the frontend, backend, tests, `Dockerfile` and CI workflow.
- [ ] **README** covering architecture, prerequisites, configuration variables, local run / test / build commands, an example request and known limitations.
- [ ] **MLOps note** (short) describing model packaging, versioning, update validation and rollback.
- [ ] **Incident notes** for the troubleshooting exercise (in the README or `INCIDENT_NOTES.md`).
- [ ] **No secrets.** No credentials, API keys or other secrets committed anywhere in the repository or its history.

The written explanations requested in section 2 (deployment triggers, environment configuration, rollback, latency investigation, operational risks) belong in the README or the MLOps note.

## 5. Optional extensions

Complete the mandatory requirements first. If you have time left, you may add any of the following:

- Deploy to a cloud service, or provide deployment configuration and a concise deployment plan.
- Use a container registry and describe a promotion flow from staging to production.
- Add structured JSON logs, request IDs, latency metrics or a small dashboard.
- Add dependency or security scanning, image scanning, or automated dependency updates.
- Add model artifact checksums, a model manifest or a simple model registry approach.

## 6. Out of scope

You do not need Kubernetes, a sophisticated model registry, distributed training, a full monitoring stack or enterprise-scale architecture. Keep the solution small, reliable and well explained.

## 7. Submission and follow-up

1. Push your solution to a repository we can access, or share it as an archive. Confirm that the repository builds and runs by following only your README.
2. Reply to the message in which you received this assessment with the link or archive, within the deadline stated there.
3. In the 15–20 minute follow-up interview, be ready to:
   - trace a request from the frontend to the model and back;
   - explain your choices on configuration, secrets, health checks and model versioning;
   - discuss what you would change before making the service production-facing.

If you want to use an alternative ML task, or anything in this document is unclear, contact us through the same channel before you start.